

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Bacharelado em Engenharia de Software
Programação Modular

Trabalho Prático — Sistema de Hospedagem StayHub
Sprint 4 — Evolução Arquitetural e Padrões de Projeto

Relatório técnico apresentado à disciplina de Programação Modular do curso de Engenharia de Software da Pontifícia Universidade Católica de Minas Gerais, referente à Sprint 4 do trabalho prático, com foco na evolução arquitetural do sistema por meio da aplicação de padrões de projeto.

Professor: Glender Brás

Belo Horizonte
2026

INTEGRANTES DO GRUPO

- Cauã Thomarco Thomaz Teixeira
- Guilherme Augusto da Silva Machado
- Sofia Figueiredo de Oliveira

SUMÁRIO

- 1 Introdução
- 2 Funcionalidades escolhidas
- 3 Solução proposta
- 4 Justificativa das escolhas
- 5 Utilização do Singleton
- 6 Benefícios obtidos com a nova arquitetura
- 7 Demonstração de funcionamento
- 8 Considerações finais
- Referências

1 INTRODUÇÃO

Este relatório documenta a Sprint 4 do trabalho prático da disciplina de Programação Modular, cujo objetivo é a evolução arquitetural do sistema de hospedagem StayHub. Nas sprints anteriores foram implementados: (i) modelagem orientada a objetos; (ii) API REST com Spring Boot; (iii) persistência em banco de dados relacional; (iv) testes automatizados com JUnit; e (v) tratamento centralizado de exceções.

Nesta etapa final o sistema é ampliado com duas novas funcionalidades, escolhidas dentre as seis opções propostas, além do uso obrigatório do padrão Singleton em pelo menos um componente global. A escolha das funcionalidades e dos padrões de projeto foi guiada por dois critérios: (a) o problema real de acoplamento e falta de extensibilidade identificados no código atual; e (b) a aderência dos padrões à natureza da variação exigida por cada requisito.

2 FUNCIONALIDADES ESCOLHIDAS

2.1 Opção 3 — Central de Notificações

O sistema passa a notificar clientes e proprietários quando eventos relevantes ocorrem no ciclo de vida de uma reserva: criação, cancelamento, check-in, check-out e confirmação de pagamento. As notificações podem ser enviadas por múltiplos canais (e-mail, SMS, WhatsApp ou notificação interna), sendo possível adicionar novos canais sem modificar o código já existente.

2.2 Opção 1 — Sistema de Tarifação Flexível

O cálculo do valor da diária passa a considerar regras contextuais como alta e baixa temporada, feriados, promoções temporárias e descontos progressivos para clientes frequentes. Novas regras podem ser criadas e combinadas livremente sem alteração significativa do código existente.

3 SOLUÇÃO PROPOSTA

3.1 Padrões de projeto utilizados

Foram aplicados quatro padrões de projeto clássicos do catálogo GoF: Observer, Strategy, Decorator e Singleton. A Tabela 1 sintetiza a aplicação de cada um.

Tabela 1 — Padrões de projeto aplicados na Sprint 4.

Padrão	Funcionalidade	Onde/como é utilizado
Observer	Central de Notificações	CentralNotificacoes publica eventos; NotificadorCliente e NotificadorProprietario são observadores registrados dinamicamente.
Strategy	Central de Notificações	Cada canal (CanalEmail, CanalSMS, CanalWhatsApp, CanalInterno) implementa CanalNotificacao.
Strategy	Tarifação Flexível	RegraTarifa é o contrato; TarifaBase é a estratégia padrão.
Decorator	Tarifação Flexível	RegraTarifaDecorator envolve outra RegraTarifa; permite empilhar AltaTemporada, Feriado, PromocaoTemporaria, etc.
Singleton	Recurso global	CentralNotificacoes, GerenciadorTarifas e LogService (uma única instância por classe).

3.2 Central de Notificações — Observer + Strategy

O problema resolvido é o acoplamento excessivo entre a lógica de negócio (AluguelService) e o mecanismo de comunicação com o mundo exterior. Sem os padrões, cada novo canal exigiria alterações em cascata no serviço, além de proliferarem condicionais para escolher qual canal disparar em qual evento. A solução aplica dois padrões complementares:

- **Observer:** a classe CentralNotificacoes (Subject) mantém uma lista de ObservadorReserva. Quando um evento ocorre no AluguelService, ele apenas publica um EventoReserva; a Central se encarrega de notificar todos os observadores interessados. Assim, o serviço não conhece os detalhes de quem escuta.
- **Strategy:** cada observador delega o envio a um CanalNotificacao injetado no construtor. Adicionar um canal Push, Telegram ou Slack no futuro exige apenas criar uma nova classe que implemente a interface, sem tocar em código existente, o que atende ao Princípio Aberto/Fechado (OCP).

3.3 Tarifação Flexível — Strategy + Decorator

O problema resolvido é a combinação dinâmica de múltiplas regras de precificação. Uma diária em 25/12 pode simultaneamente sofrer aumento por alta temporada, adicional de feriado e desconto de cliente frequente. Uma implementação com hierarquia de subclasses sofreria explosão combinatória para representar todas as combinações possíveis. A solução combina:

- **Strategy:** RegraTarifa define a interface calcular(ctx). TarifaBase apenas devolve o valor base do quarto.
- **Decorator:** RegraTarifaDecorator envolve outra RegraTarifa e aplica sua própria transformação sobre o resultado. Cada regra sazonal ou promocional é um decorator (AltaTemporada, BaixaTemporada, Feriado, PromocaoTemporaria,

DescontoClienteFrequente).

- Singleton: GerenciadorTarifas centraliza o catálogo global de regras ativas e o conjunto de feriados; monta dinamicamente a cadeia de decoradores por meio de `Function<RegraTarifa, RegraTarifa>`.

3.4 Diagrama de classes atualizado

O diagrama UML da Sprint 4 destaca as novas classes e sua relação com o núcleo pré-existente (Aluguel, AluguelService, Cliente). Ele está disponível como código PlantUML no arquivo `diagrama_sprint4.puml`, que pode ser renderizado em <https://www.plantuml.com/plantuml/uml>.

4 JUSTIFICATIVA DAS ESCOLHAS

4.1 Por que Observer e Strategy para notificações?

O comportamento desejado tem duas dimensões independentes de variação: (i) quem deve ser avisado (cliente, proprietário, futuros interessados como equipe de limpeza); e (ii) por qual canal a mensagem trafega. Observer isola a primeira dimensão; Strategy isola a segunda. Ambos são reconhecidos por Gamma et al. (2000) como soluções canônicas para esses cenários, e sua combinação permite que o sistema evolua em cada eixo sem interferência no outro.

4.2 Por que Strategy e Decorator para tarifação?

Strategy resolve o problema básico de escolher uma regra de cálculo. Contudo, o requisito impõe que múltiplas regras se apliquem simultaneamente e em ordem configurável. Decorator resolve exatamente esse tipo de composição transparente sem herança rígida, mantendo os efeitos combinados sob a mesma interface Strategy.

5 UTILIZAÇÃO DO SINGLETON

O padrão Singleton foi aplicado em três classes distintas, cada uma representando um recurso genuinamente global do sistema:

- CentralNotificacoes: barramento único de eventos. Se houvesse mais de uma instância, os observadores registrados em uma não seriam notificados por eventos publicados na outra, quebrando a semântica do padrão Observer.
- GerenciadorTarifas: mantém a lista global de feriados, promoções ativas e a cadeia de decoradores. Duas instâncias produziram cálculos divergentes para reservas idênticas.
- LogService: concentra o histórico de registros da aplicação. Um sistema de log fragmentado inviabiliza auditoria e depuração.

A Listagem 1 mostra a implementação de referência (double-checked locking),

garantindo criação preguiçosa e segura em ambientes concorrentes.

Listagem 1 — Implementação Singleton de CentralNotificacoes.

```
public class CentralNotificacoes {
    private static volatile CentralNotificacoes instancia;
    private final List<ObservadorReserva> observadores =
        Collections.synchronizedList(new ArrayList<>());

    private CentralNotificacoes() {}

    public static CentralNotificacoes getInstancia() {
        if (instancia == null) {
            synchronized (CentralNotificacoes.class) {
                if (instancia == null) instancia = new CentralNotificacoes();
            }
        }
        return instancia;
    }

    public void publicar(EventoReserva evento) { /* ... */ }
}
```

Vale destacar que o Singleton não é o único padrão da Sprint, conforme exige o enunciado: ele coexiste com Observer, Strategy e Decorator, cada um resolvendo um aspecto distinto do problema.

6 BENEFÍCIOS OBTIDOS COM A NOVA ARQUITETURA

- Extensibilidade (OCP): novos canais de notificação, novas regras de tarifação e novos observadores podem ser adicionados sem alterar código existente.
- Baixo acoplamento: AluguelService deixou de depender diretamente de e-mail, SMS ou regras de sazonalidade.
- Alta coesão: cada classe tem uma responsabilidade única (SRP).
- Facilidade de teste: as estratégias e decoradores são funções puras (ou próximas disso). Exemplo: CentralNotificacoesTest e GerenciadorTarifasTest.
- Configurabilidade em tempo de execução: promoções e feriados via REST (POST /tarifas/promocoões, POST /tarifas/feriados).
- Auditabilidade: LogService e histórico da CentralNotificacoes fornecem rastro completo (GET /notificacoes/logs, GET /notificacoes/historico).

7 DEMONSTRAÇÃO DE FUNCIONAMENTO

A demonstração das novas funcionalidades pode ser feita executando a aplicação (perfil h2 recomendado) e utilizando os endpoints REST introduzidos:

Listagem 2 — Exemplos de requisições para validar a Sprint 4.

```
# 1) Cadastrar um feriado
POST /tarifas/feriados
{ "data": "2026-12-25" }
```

```
# 2) Criar uma promocao "BlackFriday" com 30% de desconto
POST /tarifas/promocoes
{ "nome":"BlackFriday", "percentual":0.30,
  "inicio":"2026-11-20", "fim":"2026-11-30" }

# 3) Simular tarifa para uma diaria de R$ 200 em 24/12 -> 26/12
POST /tarifas/simular
{ "valorBase":200.0,
  "dataInicio":"2026-12-24", "dataFim":"2026-12-26",
  "clienteId":1 }

# 4) Consultar historico de notificacoes
GET /notificacoes/historico

# 5) Consultar log da aplicacao
GET /notificacoes/logs
```

A criação de uma reserva pelo endpoint POST /alugueis passa a: (i) calcular o valor da diária considerando as regras ativas no GerenciadorTarifas; (ii) registrar entrada no LogService; e (iii) publicar um EventoReserva do tipo RESERVA_CRIADA na CentralNotificacoes, o que aciona todos os observadores inscritos.

8 CONSIDERAÇÕES FINAIS

A Sprint 4 concluiu a evolução arquitetural do sistema StayHub, introduzindo padrões de projeto que reforçam boas práticas de desenvolvimento orientado a objetos, tais como o Princípio Aberto/Fechado, o Princípio da Responsabilidade Única e a inversão de dependência. As duas funcionalidades escolhidas — Central de Notificações e Tarifação Flexível — ilustram cenários típicos em que padrões comportamentais (Observer, Strategy) e estruturais (Decorator) trazem ganhos concretos de manutenibilidade. O uso do Singleton em três recursos globais garante coerência do estado compartilhado sem introduzir acoplamentos indesejados, uma vez que sua utilização é restrita a componentes que naturalmente demandam instância única.

REFERÊNCIAS

GAMMA, E.; HELM, R.; JOHNSON, R.; VLISSIDES, J. Padrões de projeto: soluções reutilizáveis de software orientado a objetos. Porto Alegre: Bookman, 2000.

MARTIN, R. C. Código limpo: habilidades práticas do Agile software. Rio de Janeiro: Alta Books, 2009.

FOWLER, M. Refatoração: aperfeiçoando o projeto de código existente. Porto Alegre: Bookman, 2004.

PIVOTAL. Spring Framework Documentation. 2025. Disponível em: <https://spring.io/projects/spring-framework>. Acesso em: jul. 2026.